

Cairo University
Faculty of Engineering
Computer Engineering Department

Data Structures and Algorithms

Sheet #1

Algorithms Efficiency

1. Order the following functions by growth rate:

N , N^2 , $\log N$, $N \log N$, $\log(N^2)$, $\log^2 N$, $N \log^2 N$, 2 , 2^N , 37 , $N^2 \log N$, $5 \log N$, N^3 , $10N \log N^2$

Indicate which functions grow at the same rate.

2. For each of the following program fragments, estimate the order of time complexity.

a) <code>count=0; for(i=0; i<n; i++) count++;</code>	b) <code>count=0; for(i=0; i<n; i+=2) count++;</code>
c) <code>count=0; for(i=1; i<n; i*=2) count++;</code>	d) <code>count=0; for(i=1; i<n; i*=2) count++; for(i=0; i<n; i+=2) count++;</code>
e) <code>count=0; for(i=0; i<n; i++) for(j=0; j<n; j++) count++;</code>	f) <code>count=0; for(i=0; i<n; i++) for(j=1; j<n; j*=2) count++;</code>
g) <code>count=0; for(i=0; i<n; i++) for(j=0; j<i; j++) count++;</code>	h) <code>count=0; for(i=0; i<n; i++) for(j=0; j<n*n; j++) count++;</code>
i) <code>count=0; for(i=0; i<n; i++) for(j=1; j<n; j*=2) count++; for(i=0; i<n; i++) count++;</code>	j) <code>count=0; for(i=0; i<n; i++) for(j=0; j<i*i; j++) for(k=0; k<j; k++) count++;</code>
k) <code>count=0; for(i=n/2; i<=n; i++) for(j=1; j+n/2<=n; j++) for(k=1; k<=n; k*=2) count++;</code>	l) <code>i=1,s=1; while(s<=n) { i++; s+=i; }</code>

3. In each of the following problems, you are given two algorithms that do the same job. You are required to indicate which of the two algorithms is “**better**” and why.
- a. The following two algorithms count the number of values that are above the average of a given set of values.

Algorithms: AboveAvg1 & AboveAvg2

Input → L: List of integer values, N: size of the List

Output → C: number of values above average of all values

AboveAvg1 (L, N)	AboveAvg2 (L, N)
<pre> 1. Set Avg=0, Total = 0, C = 0 2. For I = 1 to N Set Total = Total + L[I] 3. Set Avg = Total/N 4. For I = 1 to N 1- IF (L[I] > Avg) Set C = C + 1 5. Return C </pre>	<pre> 1. Set Avg = 0, C = 0 2. For J = 1 to N 1- set Total = 0 2- For I = 1 to N Set Total = Total + L[I] 3- Set Avg = Total/N 4- IF (L[J] > Avg) Set C = C + 1 3. Return C </pre>

- b. The following two algorithms take a list L and output another list M such that M contains the even numbers of L first then odd numbers of L.

Algorithms: EvensFirst1 & EvensFirst2

Input → L: List of integer values, n:size of List L

Output → M: List of integers such that even numbers are first then odd numbers

EvensFirst1 (L, n, M)	EvensFirst2 (L, n, M)
<pre> 1. J = 1 2. For I = 1 to n 1- IF(L[I] % 2 == 0) //Even a. M[J] = L[I] b. J = J+1 3. For I = 1 to n 2- IF(L[I] % 2 != 0) //Odd a. M[J] = L[I] b. J = J+1 4. Return M </pre>	<pre> 1. J = 1 2. K = Size 3. For I = 1 to n 1- IF(L[I] % 2 == 0) //Even a. M[J] = L[I] b. J = J+1 2- ELSE //Odd a. M[K] = L[I] b. K = K - 1 4. Return M </pre>

4. Algorithms **A** and **B** spend exactly $T_A(n) = 0.1 * n^2 * \log_{10}(n)$, and $T_B(n) = 2.5 * n^2$ microseconds, respectively, for a problem of size n . Choose the algorithm, which is better in the Big-Oh sense, and find out a problem of size n_0 such that for any larger size $n > n_0$ the chosen algorithm outperforms the other. If your problems are of size $n \leq 10^9$, which algorithm will you recommend to use.
5. Software packages **A** and **B** of complexity $O(n * \log(n))$ and $O(n)$, respectively, spend exactly $T_A(n) = C_A * n * \log_{10}(n)$, and $T_B(n) = C_B * n$ milliseconds to process n data items. During a test, the average time of processing $n = 10^4$ data items with the package **A** and **B** is 100 milliseconds and 500 milliseconds, respectively. Work out the exact condition when one package actually outperforms the other and recommend the best choice if up to $n = 10^9$ items should be processed.